

BSE3211 UNIT TESTING - INFORMAL TEST PLAN BY GROUP DAJ

NAME	STUDENT NUMBER	REGISTRATION NUMBER
KARAGWA ANN TREASURE	23/U/09034/PS	2300709034
WANYANA SELINA MASEMBE	23/U/1529	2300701529
AYAN MUSTAFA ABDIRAHMAN	23/X/22948/EVE	2300722948
AMABE JUNIOR HUMPHREY	23/U/05942/PS	2300705942
AKANGA ANDREW	23/U/05555/PS	2300705555

1. Objective

The purpose of this testing exercise is to verify the correctness, robustness, and reliability of the Meeting Planner system. The system supports scheduling meetings, booking vacations, checking availability, and printing agendas for both people and rooms.

The tests aim to confirm: - Correct handling of valid operations - Proper rejection or handling of invalid inputs - Accurate detection of scheduling conflicts - Correct behavior of exception handling - System stability under edge cases and boundary conditions

2. Scope of Testing

The following components were tested: - **Calendar** - core scheduling logic - **Person** - user-level interaction with calendar - **Meeting** - meeting representation and data handling - **Room** - room-level calendar delegation - **Organization** - lookup of default employees and rooms - **TimeConflictException** - custom exception handling

3. Test Strategy

Unit testing with JUnit; white-box tests exercised the classes directly. The approach covered:

A. Normal Functionality Tests

- Adding valid meetings
- Checking availability
- Printing agendas
- Removing meetings

- Looking up known employees and rooms

B. Boundary Value Tests

- Back-to-back meetings where one meeting ends when another begins
- Full-day meetings with start time 0 and end time 23
- Edge time values including invalid lower and upper hour values
- Zero-length meetings where start time equals end time

C. Invalid Input Tests

- Invalid days, such as day 35
- Invalid months, such as month 13
- Invalid time ranges, such as start time after end time
- Invalid start and end hours, such as -1 and 24
- Unknown employee and room lookups

D. Conflict Detection Tests

- Overlapping room meetings
- Double booking the same time slot
- Vacation or full-day meetings conflicting with timed meetings

E. Exception Handling Tests

- Ensure `TimeConflictException` is thrown for conflicts and invalid scheduling input
- Verify room-level exception messages include room context
- Verify organization lookup failures return clear messages

4. Test Cases Summary

CalendarTest.java

- `testAddMeeting_holiday` - add full-day holiday, day becomes busy
- `testAddMeeting_normalTime` - add timed meeting, `isBusy` returns true
- `testAddMeeting_conflict` - same time twice throws `TimeConflictException`
- `testAddMeeting_overlap` - overlapping times throw exception
- `testAddMeeting_backToBack` - current implementation treats back-to-back meetings as a conflict
- `testAddMeeting_invalidTime` - invalid hour rejected
- `testAddMeeting_invalidMonth` - month 13 rejected
- `testAddMeeting_invalidDay` - behavior observed for invalid day values
- `testAddMeeting_invalidTimeRange` - start after end rejected
- `testClearSchedule` - clearing a day removes meetings
- `testRemoveMeeting` - removing a meeting frees the slot
- `testDecemberBug` - regression test for December handling

PersonTest.java

- **testGetName** - name stored correctly
- **testAddMeeting_success** - person becomes busy after adding meeting
- **testAddMeeting_conflictMessageIncludesName** - conflict message contains person name
- **testPrintAgenda_day** - daily agenda contains meeting description
- **testPrintAgenda_month** - monthly agenda contains meeting description
- **testIsBusy_falseWhenFree** - person is free when no meetings exist
- **testRemoveMeeting** - removal frees time
- **testGetMeeting** - retrieve meeting and description

MeetingTest.java

- **testFullDayConstructorSetsDateAndAllDayTimes** - the full-day constructor stores the requested month/day and sets the meeting from 0 to 23.
- **testFullDayConstructorWithDescriptionSetsDescription** - the full-day constructor with description also stores the supplied description.
- **testTimedConstructorSetsDateAndTimes** - the timed constructor stores month, day, start time, and end time.
- **testDetailedConstructorSetsAllMeetingFields** - the detailed constructor stores the supplied attendee list, room object, description, date, and times.
- **testSettersUpdateMeetingFields** - setters update month, day, start time, end time, room, and description.
- **testAddAndRemoveAttendee** - attendees can be added and removed from a meeting-backed attendee list.
- **testToStringFormatsMeetingDetails** - **toString()** includes date, time range, room ID, description, and comma-separated attendee names.

OrganizationTest.java

- **testDefaultConstructorCreatesExpectedEmployees** - the organization initializes exactly five default employees: Namugga Martha, Shema Collins, Acan Brenda, Kazibwe Julius, and Kukunda Lynn.
- **testDefaultConstructorCreatesExpectedRooms** - the organization initializes exactly five default rooms: LLT6A, LLT6B, LLT3A, LLT2C, and LAB2.
- **testGetEmployeeReturnsMatchingEmployee** - lookup by an existing employee name returns the correct **Person**.
- **testGetRoomReturnsMatchingRoom** - lookup by an existing room ID returns the correct **Room**.
- **testGetEmployeeThrowsExceptionForUnknownEmployee** - unknown employee lookup throws an exception with message **Requested employee does not exist**.

- `testGetRoomThrowsExceptionForUnknownRoom` - unknown room lookup throws an exception with message `Requested room does not exist`.

`RoomTest.java`

- `testDefaultConstructorSetsEmptyID` - the default constructor creates a room with an empty ID.
- `testConstructorSetsRoomID` - the ID constructor stores the supplied room ID.
- `testAddMeetingMakesRoomBusy` - adding a valid meeting makes the room busy during that meeting's time range.
- `testGetMeetingReturnsAddedMeeting` - `getMeeting()` returns the same meeting object that was added to the room calendar.
- `testRemoveMeetingMakesRoomAvailable` - removing a meeting frees the room for the removed time slot.
- `testPrintAgendaForDayIncludesMeeting` - the room daily agenda includes the scheduled meeting details and attendees.
- `testAddMeetingThrowsRoomConflictMessageForOverlap` - overlapping meetings throw `TimeConflictException`, and the message identifies the room and existing conflicting meeting.
- `testIsBusyThrowsExceptionForInvalidTime` - checking an invalid time range throws `TimeConflictException` with message `Meeting starts before it ends`.
- `testAddMeetingThrowsExceptionForInvalidDay` - adding a meeting on invalid day 35 throws a room-scoped conflict message containing `Day does not exist`.
- `testAddMeetingThrowsExceptionForInvalidMonth` - adding a meeting in invalid month 13 throws a room-scoped conflict message containing `Month does not exist`.
- `testAddMeetingThrowsExceptionForInvalidStartHour` - adding a meeting with start hour -1 throws a room-scoped conflict message containing `Illegal hour`.
- `testAddMeetingThrowsExceptionForInvalidEndHour` - adding a meeting with end hour 24 throws a room-scoped conflict message containing `Illegal hour`.
- `testAddMeetingThrowsExceptionWhenStartEqualsEnd` - adding a zero-length meeting throws a room-scoped conflict message containing `Meeting starts before it ends`.

`TimeConflictExceptionTest.java`

- `testDefaultConstructor` - null message
- `testMessageConstructor` - message stored
- `testCauseConstructor` - cause stored
- `testMessageAndCauseConstructor` - message and cause stored
- `testFullConstructor` - full-parameter constructor behavior

5. Key Findings

Finding 1: Meeting Constructors Correctly Preserve Supplied Data

- Source: `MeetingTest.java`
- Observation: full-day, timed, and detailed constructors store the expected date, time, attendees, room, and description values.
- Impact: meeting creation works correctly for the valid construction paths covered by the tests.
- Severity: Positive
- Recommendation: keep these constructor tests because they protect the basic meeting data model.

Finding 2: Meeting Setter Methods Work for Core Mutable Fields

- Source: `MeetingTest.java`
- Observation: setters update month, day, start time, end time, room, and description.
- Impact: code that creates a blank `Meeting` and fills it later can rely on the tested setters.
- Severity: Positive
- Recommendation: add a setter or constructor path for attendees if blank meetings are expected to support attendee operations.

Finding 3: Meeting Attendee Operations Depend on a Non-null Attendee List

- Source: `MeetingTest.java`
- Observation: `testAddAndRemoveAttendee` passes because the meeting is created with an initialized `ArrayList<Person>`.
- Impact: `addAttendee()` and `removeAttendee()` are functional only when `attendees` has already been initialized.
- Severity: Medium
- Recommendation: initialize `attendees` in the default and partial constructors, or guard against null before modifying attendees.

Finding 4: `Meeting.toString()` Formatting is Verified but Null-Sensitive

- Source: `MeetingTest.java`
- Observation: the format is verified as `month/day, start - end,roomID:description` followed by an attendee list.
- Impact: the method is reliable for complete meeting objects, but it assumes `room` and `attendees` are non-null.
- Severity: Medium
- Recommendation: either require complete meeting objects before printing agendas or make `toString()` null-safe.

Finding 5: Organization Default Data is Stable and Test-Covered

- Source: `OrganizationTest.java`
- Observation: the tests confirm five default employees and five default rooms in a specific order.
- Impact: the current organization seed data is protected against accidental removal or reordering.
- Severity: Positive
- Recommendation: keep these tests if the default data is part of the required specification; loosen order checks if order should not matter.

Finding 6: Organization Lookup Behavior is Clear for Known and Unknown Values

- Source: `OrganizationTest.java`
- Observation: valid employee and room lookups return matching objects, while unknown lookups throw exceptions with clear fixed messages.
- Impact: callers receive predictable behavior from the lookup API.
- Severity: Positive
- Recommendation: consider replacing generic `Exception` with a more specific exception type if the API evolves.

Finding 7: Room Correctly Delegates Scheduling to Its Calendar

- Source: `RoomTest.java`
- Observation: adding, retrieving, printing, and removing meetings through `Room` behave as expected.
- Impact: room-level scheduling works for normal valid meeting flows.
- Severity: Positive
- Recommendation: keep room tests because they verify the delegation boundary between `Room` and `Calendar`.

Finding 8: Room Conflict Messages Preserve Room Context

- Source: `RoomTest.java`
- Observation: room scheduling errors are wrapped with messages such as `Conflict for room LLT6A`, while preserving the underlying conflict detail.
- Impact: users and developers can identify which room caused a scheduling failure.
- Severity: Positive
- Recommendation: continue asserting message content for room conflicts because it is useful user-facing behavior.

Finding 9: Room Invalid Input Handling Covers Multiple Edge Cases

- Source: `RoomTest.java`

- Observation: invalid day, invalid month, illegal start hour, illegal end hour, reversed time ranges, and zero-length meetings all produce `TimeConflictException`.
- Impact: the room API rejects many invalid scheduling requests before they corrupt the schedule.
- Severity: Positive
- Recommendation: add equivalent coverage at the `Person` or `Calendar` level if the same validation behavior is expected across all scheduling entry points.

Finding 10: Back-to-Back Meeting Handling May Be Too Strict

- Source: `CalendarTest.java`
- Observation: current tests document that a meeting ending at the same hour another starts is treated as a conflict.
- Impact: this blocks common scheduling scenarios where one meeting immediately follows another.
- Severity: Medium
- Recommendation: decide whether end times should be inclusive or exclusive. If end times are exclusive, update overlap logic and expected tests.

6. Overall Assessment

The Meeting Planner implements core scheduling, lookup, conflict detection, and agenda printing behavior with useful unit coverage. The requested files add confidence that meeting data is stored correctly, default organization data is available, and room scheduling rejects invalid or conflicting requests.

The main remaining risks are null-safety in partially initialized `Meeting` objects, reliance on exact seeded data order in `Organization`, and the strict interpretation of meeting time boundaries.

Why PlannerInterface was not unit tested

The `PlannerInterface` class was not extensively unit tested because it mainly handles user input and output, rather than core application logic.

It depends heavily on:

- Console input/output (`System.in`, `System.out`)
- Interactive menu navigation
- Continuous user-driven execution

Because of this, its behavior is not deterministic, making it unsuitable for standard unit testing, which requires isolated and predictable logic.

Proper testing would also require mocking input/output streams, which is beyond the scope of this assignment.

Instead, unit testing focused on core logic classes such as **Calendar**, **Person**, **Meeting**, **Organisation** and **Room** which contain testable business rules.

7. Conclusion

The system is suitable for basic scheduling flows. To improve robustness, the next priorities should be initializing **Meeting** collections consistently, making agenda formatting safer for partial meetings, and clarifying whether back-to-back meetings should be allowed.

8. Test Files Location

All tests referenced are in `meetingplanner/src/test/java/edu/sc/bse3211/meetingplanner/`.

The project can be accessed from the github project link: <https://github.com/Karagwa/meetingplanner>

Run them from the Maven project directory:

```
cd meetingplanner
mvn test
```